Escola Profissional
BENTO DE JESUS CARAÇA
DELEGAÇÃO DE LISBOA

RELATÓRIO

Desenvolvimento de um Sistema de Gestão de Biblioteca em Python utilizando Classes e Objetos

CURSO TÉCNICO DE PROGRAMAÇÃO E GESTÃO DE SISTEMAS INFORMÁTICOS

Escola: Escola Profissional Bento de Jesus Caraça

Professor : Breno Sousa

Nome dos Alunos: Núria Melo, Zilmara Paulo, Murilo Aguilar, Marco Piedade, Rodrigo Ribeiro

Nº Aluno: L2624, L2623, L2667, L2669, L2666

30/10/2026

O relatório encontra-se em condições para ser apresentado

Sim

Ciclo de Formação 2025/2028

Ano Letivo 2026/2027

INDICE

Introdução 3

Capítulo I · Sobre o Trabalho 4

Capítulo II · Atividades Desenvolvidas e Justificação Técnica 5

Capítulo III · Cronograma do Projeto 11

Capítulo IV · Menu 13

Conclusão 14

INTRODUÇÃO

Este trabalho, consiste no desenvolvimento de um sistema de gestão de biblioteca utilizando a linguagem de programação Python e o uso de classes e objetos.

O trabalho desenvolvido tem como principal objetivo criar um sistema de que facilite a gestão de uma biblioteca escolar permitindo registar novos livros e alunos, consultas livros disponíveis, fazer empréstimo de livros, devolver livros e mostrar os livros emprestados pelos alunos.

Durante a realização do trabalho foram aplicados em prática os conhecimentos das aulas, principalmente a programação orientada em objetos através da utilização de classes.

Com esse trabalho, pretende-se desenvolver uma solução que contribua para um sistema simples para gerenciar o empréstimo de livros da

Biblioteca da escola.

CAPÍTULO I –- Sobre o Trabalho

O trabalho foi realizado na escola profissional Bento de Jesus Caraça na delegação de Lisboa, e também em casa. Foram dadas 8 horas de aulas para realizar o trabalho.

Este trabalho consiste no desenvolvimento de um sistema de gestão de biblioteca, com objetivo em ajudar uma escola que precisa de um sistema simples para gerenciar o empréstimo de livros da sua biblioteca.

O sistema permite realizar diferentes operações tais como, registar novos livros e alunos, consultas livros disponíveis, fazer empréstimo de livros, devolver livros e mostrar os livros emprestados por cada aluno através da linguagem de Python e uso de classes/objetos. Foram impostos alguns requisitos no desenvolvimento do trabalho os quais, criação de 3 classes e as suas constituições.

O desenvolvimento deste projeto permite também aplicar os conhecimentos adquiridos ao longo das aulas e desenvolver competências na área da programação.

Capítulo II – Atividades Desenvolvidas durante o período do projeto e sua justificação Técnica

O trabalho realizado durante as aulas consistiu na criação de 3 classes: Livro, Aluno e Biblioteca. E cada classe foi desenvolvida com uma função especifica. O método __init __() é executado quando um objeto de uma classe é criado e quem recebe o título, o autor e código e guarda esses valores no objeto, o título guarda o título o autor guarda a autor e o self representa o próprio objeto que está a ser criado.

A classe livro guarda o título, o autor, o código e a disponibilidade do livro (se esta disponível ou não).

Nesta classe, o método mostrar_dados() apresenta as informações principais do livro: o código, o título, o autor e o seu estado. Para indicar a disponibilidade, o método verifica o valor do atributo disponivel. Se for True, o livro aparece como disponível; caso contrário, aparece como emprestado.

A classe Aluno possui os atributos nome, matricula e livros_emprestados. Este último é uma lista que começa vazia e que vai guardando os objetos Livro associados ao aluno quando este faz um empréstimo. O método mostrar_dados() apresenta a matrícula, o nome e a quantidade de livros emprestados. Para contar os elementos da lista é utilizada a função len().

```
class Livro:
    def __init__(self, titulo, autor, codigo):
        self.titulo = titulo
        self.autor = autor
        self.codigo = codigo
        self.disponivel = True
```

Depois de criar a classe livro cria-se as suas determinadas funções.

```
(method) def registar_livro(self: Self@Biblioteca) -> None
def registar_livro(self):
    titulo = input("Título do livro: ").strip()
    autor = input("Autor: ").strip()
    if titulo == "" or autor == "":
        print("Título e autor não podem estar vazios.")
        return
    codigo = len(self.catalogo_livros) + 1
    novo_livro = Livro(titulo, autor, codigo)
    self.catalogo_livros.append(novo_livro)
    print(f"Livro registado com o código {codigo}.")
```

```
def consultar_livros_disponiveis(self):
    encontrou = False
    for livro in self.catalogo_livros:
        if livro.disponivel:
            print(livro.mostrar_dados())
            encontrou = True
    if not encontrou:
        print("Não há livros disponíveis.")


def devolver_livro(self):
    try:
        matricula = int(input("Matrícula do aluno: "))
    except ValueError:
        print("Erro! Escreva só números.")
        return

    aluno = self.procurar_aluno(matricula)
    if aluno is None:
        print("Aluno não encontrado.")
        return
    if not aluno.livros_emprestados:
        print("Este aluno não tem livros emprestados.")
        return

    print("Livros emprestados a este aluno:")
    for livro in aluno.livros_emprestados:
        print(livro.mostrar_dados())

    try:
        codigo = int(input("Código do livro a devolver: "))
    except ValueError:
        print("Erro! Escreva só números.")
        return

    for livro in aluno.livros_emprestados:
        if livro.codigo == codigo:
            livro.disponivel = True
            aluno.livros_emprestados.remove(livro)
            print(f"O livro '{livro.titulo}' foi devolvido.")
            return
    print("Esse livro não está emprestado a este aluno.")
```

```python
def mostrar_livros_do_aluno(self):
    try:
        matricula = int(input("Matrícula do aluno: "))
    except ValueError:
        print("Erro! Escreva só números.")
        return

    aluno = self.procurar_aluno(matricula)
    if aluno is None:
        print("Aluno não encontrado.")
    elif not aluno.livros_emprestados:
        print(f"{aluno.nome} não tem livros emprestados.")
    else:
        print(f"Livros emprestados a {aluno.nome}:")
        for livro in aluno.livros_emprestados:
            print(livro.mostrar_dados())

def estado_livro(self):
    total = len(self.catalogo_livros)
    emprestados = 0
    for livro in self.catalogo_livros:
        if not livro.disponivel:
            emprestados += 1
    print("estado de livro")
    print(f"Total de livros: {total}")
    print(f"Livros emprestados: {emprestados}")
    print(f"Livros disponíveis: {total - emprestados}")
```

```
def emprestar_livro(self):
    if not self.catalogo_livros:
        print("Não há livros registados.")
        return
    if not self.cadastro_alunos:
        print("Não há alunos registados.")
        return

    print("Livros disponíveis:")
    self.consultar_livros_disponiveis()

    try:
        matricula = int(input("Matrícula do aluno: "))
        codigo = int(input("Código do livro: "))
    except ValueError:
        print("Erro! Escreva só números.")
        return

    aluno = self.procurar_aluno(matricula)
    livro = self.procurar_livro(codigo)

    if aluno is None:
        print("Aluno não encontrado.")
    elif livro is None:
        print("Livro não encontrado.")
    elif not livro.disponivel:
        print("Esse livro já está emprestado.")
    else:
        livro.disponivel = False
        aluno.livros_emprestados.append(livro)
        print(f"O livro '{livro.titulo}' foi emprestado a {aluno.nome}.")
```

Já a classe Aluno regista o nome, a matricula, e os livros que tem emprestados.

```
class Aluno:
    def __init__(self, nome, matricula):
        self.nome = nome
        self.matricula = matricula
        self.livros_emprestados = []
```

E por último a classe biblioteca permite organizar o catálogo de livros e registo dos alunos, além de realizar empréstimos, devoluções e consultas.

A classe Biblioteca contém duas listas: catalogo_livros, onde ficam guardados os livros registados, e cadastro_alunos, onde ficam guardados os alunos. É nesta classe que se encontram os métodos responsáveis por gerir as operações da biblioteca.

O método registar_livro() pede o título e o autor, verifica se os campos foram preenchidos e cria um novo objeto da classe Livro. O código é atribuído automaticamente através da quantidade de livros já

existentes na lista. O método registar_aluno() funciona de forma semelhante: pede o nome, verifica se não está vazio e atribui uma matrícula automática.

Para localizar os registos, foram criados os métodos procurar_livro() e procurar_aluno(). Estes percorrem as respetivas listas e comparam o código do livro ou a matrícula do aluno com o valor indicado. Quando encontram o elemento, devolvem o objeto correspondente; se não o encontrarem, devolvem None.

O método consultar_livros_disponiveis() percorre o catálogo e apresenta apenas os livros cujo atributo disponivel está como True. Uma variável chamada encontrou permite verificar se foi apresentado algum livro; se nenhum estiver disponível, é mostrada uma mensagem ao utilizador.

No método emprestar_livro(), o programa começa por confirmar que existem livros e alunos registados. Depois pede a matrícula do aluno e o código do livro. O programa procura os dois registos e verifica se existem e se o livro está disponível. Se todas as condições forem cumpridas, altera a disponibilidade do livro para False e acrescenta o objeto à lista de livros emprestados do aluno.

O método devolver_livro() pede a matrícula do aluno e apresenta os livros que esse aluno tem emprestados. Depois solicita o código do livro a devolver. Se o livro pertencer à lista desse aluno, a sua disponibilidade passa novamente para True e o objeto é retirado da lista de empréstimos. Caso contrário, o programa informa que o livro não está emprestado a esse aluno.

O método mostrar_livros_do_aluno() permite consultar os livros associados a uma matrícula. Se o aluno existir e tiver empréstimos, são apresentados os dados de cada livro. Se não existir ou não tiver livros emprestados, o programa apresenta a mensagem correspondente.

Por fim, o método estado_livro() apresenta um pequeno relatório com o total de livros registados, quantos estão emprestados e quantos

continuam disponíveis. Para calcular os empréstimos, percorre o catálogo e conta os livros cujo atributo disponível está como false.

```
class Biblioteca:
    def __init__(self):
        self.catalogo_livros = []
        self.cadastro_alunos = []
```

Para desenvolver o trabalho foi utilizado o Python, a aplicação visual code, live share e um computador. Também foram utilizados conceitos de programação como listas, ciclos for, condições if/elif/else, métodos, criação de objetos e tratamento de erros com try/except. O tratamento de ValueError é utilizado quando o utilizador introduz um valor que deveria ser numérico, evitando que o programa termine inesperadamente.

A utilização de strip() na leitura dos textos permite retirar espaços em branco no início e no fim da informação introduzida. As verificações feitas antes de registar livros e alunos impedem a criação de registos com campos obrigatórios vazios.

Durante o trabalho foram realizadas várias atividades, primeiro a criação

das classes, depois as funcionalidades para registar livros e alunos, também foi implementado um menu.

E por último dedicou-se à correção de erros e à preparação do relatório técnico.

Capitulo III - Cronograma do projeto

CRONOGRAMA DO TRABALHO ESCOLAR — SEMANA 1					
Horário	**Segunda-feira**	**Terça-feira**	**Quarta-feira**	**Quinta-feira**	**Sexta-feira**
08:30–09:30					Criação das classes
09:40–10:20					Fazer o Menu
10:40–11:30					
11:40–12:30					
12:40–13:30					
13:40–14:30					
14:50–15:40		Apresentação do trabalho a turma			
15:50–16:40		Planeamento do projeto			

CRONOGRAMA DO TRABALHO ESCOLAR — SEMANA 2					
Horário	**Segunda-feira**	**Terça-feira**	**Quarta-feira**	**Quinta-feira**	**Sexta-feira**
08:30–09:30					Testar o codigo e corrigir os erros
09:40–10:20					Começar o Relatório
10:40–11:30					
11:40–12:30					
12:40–13:30					
13:40–14:30					
14:50–15:40		Implementação do empréstimo de livros e da associação dos livros aos alunos.			
15:50–16:40					

CRONOGRAMA DO TRABALHO ESCOLAR — SEMANA 1					
Horário	**Segunda-feira**	**Terça-feira**	**Quarta-feira**	**Quinta-feira**	**Sexta-feira**
08:30–09:30					Apresentação do trabalho
09:40–10:20					
10:40–11:30					
11:40–12:30					
12:40–13:30					
13:40–14:30					
14:50–15:40		Finalização do trabalho, correçoes de erros e finalizar o relatorio			
15:50–16:40					

Capítulo IV – Menu

Após a realização de todo o código de criar as classes com os seus respetivos métodos,
seguimos com a realização do menu que permite ao usuário escolher aquilo que pretende, em baixo na imagem conseguimos visualizar como aparecem as opções.

```
while True:
    print("   Menu   ")
    print("1 - Registar aluno")
    print("2 - Registar livro")
    print("3 - Consultar livros disponíveis")
    print("4 - Emprestar livro")
    print("5 - Devolver livro")
    print("6 - Livros emprestados por um aluno")
    print("7 - Sair")

    try:
        opc = int(input("Escolha uma opção: "))
    except ValueError:
        print("Erro! Escreva só números.")
        continue

    if opc == 1:
        biblioteca.registar_aluno()
    elif opc == 2:
        biblioteca.registar_livro()
    elif opc == 3:
        biblioteca.consultar_livros_disponiveis()
    elif opc == 4:
        biblioteca.emprestar_livro()
    elif opc == 5:
        biblioteca.devolver_livro()
    elif opc == 6:
        biblioteca.mostrar_livros_do_aluno()
    elif opc == 7:
        biblioteca.estado_livro()
        print("Encerrou o programa.")
        break
    else:
        print("Opção inválida, escolha um número de 1 a 7.")
```

Capítulo IV - CONCLUSÃO

Neste trabalho foi possível utilizar a linguagem de Python e o uso de classes/objetos. O desenvolvimento deste projeto permitiu também aplicar os conhecimentos adquiridos ao longo das aulas e desenvolver competências na área da programação. Também é importante ressaltar que o trabalho também serviu para aprimorar o trabalho em grupo, com desafios para nós desvendarmos.